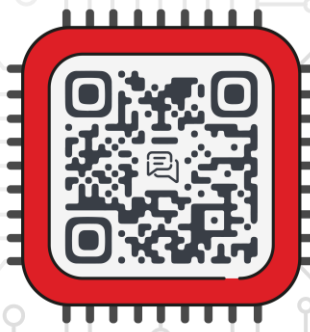




3

14

1680 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

Multithreading in java

د. محمد الأحمد



برمجة ٣

RB Informatics; 30/10/2024

سنتعرف في البداية عن مفهوم ال process

■ Process : program in execution

أي أن البرنامج عندما يبدأ بالتنفيذ يسميه ال Operating system بال process ومعظم أنظمة التشغيل الحديثة تسمح بتنفيذ أكثر من process في وقت واحد

■ مثال : من الممكن استخدام ال word ونحن نستمتع إلى الموسيقى أي يمكن تشغيل عدة برامج في نفس الوقت، نزن أن هذه البرامج يتم تشغيلها جميعها في ذات الوقت معاً ولكن في الحقيقة يتم التبديل بين process وأخرى (switching) بشكل سريع جداً لا نلاحظه، وهذا ما يسمى بال multitasking.

■ ما هو مفهوم ال Multitasking ؟

■ Multitasking operating systems run multiple programs simultaneously.

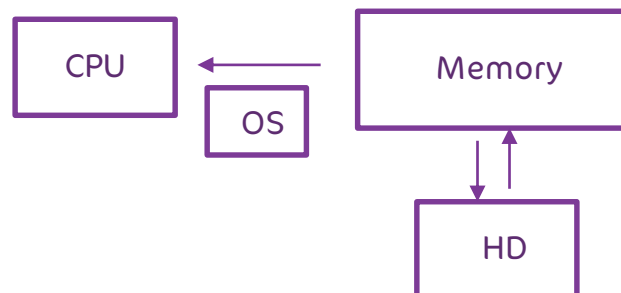
■ أي يتم تنفيذ أكثر من task في آن معاً بشكل متزامن concurrency (في الحقيقة هناك process واحد يعمل ولكن يتم التبديل بين ال processes بشكل سريع -switching-).

وهي تتحقق من خلال :

Multiprocessing

Multithreading والتي سنتعرف عليها في هذه المحاضرة

■ مثال عن آلية تنفيذ ال process :



- نعلم أن أي برنامج يخزن على الهارد ديسك (HD) وبمجرد الضغط عليه يتم نقله إلى Memory وفي الذاكرة من الممكن تواجد أكثر من program، و في ال CPU يتم تنفيذ البرنامج و لكن يأخذ بعض الوقت ليتنفذ ، و تأتي أهمية ال Operating system حيث يتحكم بترتيب تنفيذ البرامج حسب خوارزمية معينة.

(البرمجة على التوازي) Parallelism	(البرمجة على التزامن) Concurrency
• تقسيم ال processor (المعالج) إلى عدة نويات (cores)، عندها يتم تنفيذ أكثر من process واحدة في ذات الوقت فعلاً أي على التوازي، والهدف منها تسريع إنجاز المهمة. • كل نواة تعالج process بشكل مستقل و يعد ال Multithreading نوعاً منها	• يتم التعامل مع أكثر من process ظاهرياً في نفس الوقت ولكن في الحقيقة يتم التبديل بسرعة بين المهام بحيث تبدو أنها تعمل في وقت واحد. • أكثر من process تُنفذ على النواة ذاتها (ال core أو ال processor (المعالج) ذاته) بشكل متزامن ولكن لا تنفذ في الوقت ذاته.

- الهدف من ال parallelism و ال Multithreading تحسين الأداء و السرعة في تنفيذ المهام والاستخدام الأعظم لل CPU أي (CPU) to maximize the utilization of the processor.

- أي باختصار :

Parallelism	Concurrency
When tasks literally run at the same time	When two or more tasks can start , run in overlapping time periods

ما هو ال Thread ؟

- Smallest unite of execution/path of execution أي هو أصغر وحدة في التنفيذ أو هو مسار التنفيذ (مجموعة تعليمات) و ال thread يعتبر lightweight

مثال عن ال thread :

- عند فتح برنامج ال word (هو process) ولكن يستطيع المستخدم في نفس الوقت الكتابة عليه ويقوم البرنامج بفحص الكلمات اذا كانت تحتوي على أخطاء املائية أي استطعنا :

Writing وهو يمثل thread

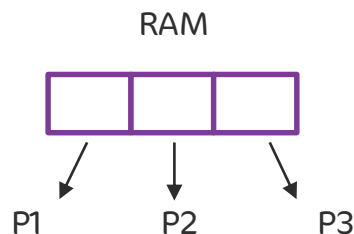
Check وهو أيضاً يمثل thread آخر

- أي نستنتج من المثال السابق أنه لا يوجد thread من دون process أي: (at least one process is required for each thread)

- Word (process)
 - (thread 1) writing
 - (thread 2) check
- نلاحظ أنه يتم تنفيذ أجزاء من البرنامج في آن معاً.

ملاحظات :

- Program: not active in HD
- بإمكاننا معرفة التطبيقات التي تعمل معاً من خلال الدخول الى task manager من قائمة البدء أو من خلال الضغط على ctrl+shift+Esc
- الفرق بين ال process و ال Thread في الذاكرة :
نلاحظ أن كل process تأخذ منطقة مستقلة (space) في ال RAM و بما أن كل process هو فعلياً يمثل program عند التنفيذ فهو يأخذ منطقة مستقلة بحد ذاته بحيث أنه إذا تعطل ال process لسبب ما فإنه لن يؤثر على بقية ال processes الموجودة في ال RAM ، و بما أن أنظمة التشغيل الحديثة تدعم عملية Switching بين ال processes وهي تعمل من خلال الانتقال من فضاء ال process الأولى إلى فضاء ال (Memory) آخر، فإن هذه العملية أصبحت مرهقة (heavyweight) على ال operating system.



- Threads within a process share the same address space (ال threads ضمن نفس ال process تأخذ مكاناً واحداً في الذاكرة).
- كل process مؤلفة من threads ، وال threads ضمن ال process ذاتها تحجز ذات الفضاء في الذاكرة، مما يجعل عملية ال switching سهلة جداً لأن ال threads موجودة في فضاء ذاكرة واحد.

- مفهوم ال Multithreading يتحقق بال Thread أو بال process

ما هي ال Multithreading ؟

- More than one thread in execution (أكثر من ثريد ينفذ في آن معاً ظاهرياً ولكن في الحقيقة يتم التبديل بينها بسرعة فتبدوا لنا وكأنها تنفذ في ذات الوقت، أي تنفذ على التزامن)

- وهي ميزة مهمة تستخدم في برمجة الألعاب حيث يعتبر كل لاعب thread أو كل سلاح هو عبارة عن thread ، وأيضا تستخدم في تطبيقات الويب عند فتح موقع ما يحتوي على content مختلفة، يتم بداية تحميل ال text ثم الصور ومن ثم الفيديوهات حيث يعتبر كل جزء منهم thread.
- أي برنامج مكتوب بلغة جافا عند التنفيذ يعتبر process يوجد فيها حتماً thread يسمى main thread يتم إنشاؤه تلقائياً (هو مجموعة تعليمات)، ممكن أن يتقسم إلى مجموعة من threads.
- ونلاحظ عند رمي exception أن المشكلة قد حصلت في ال main thread.

```
public class Main{
public static void main(String[] args) {
    /*
    يوجد خطأ في الكود و هو أننا قمنا بتعريف المتغير
    ووضعنا فيه نصاً بدل أن نضع فيه رقم int ك x هنا
    */
    int x = "i am a string!";
}
}
```

Output: Exception in thread "main" java.lang.RuntimeException:Uncompilable source code-incompatible types
required :int
found:java.lang.String

Multitasking

■ مقارنة بين :

Multiprocessing	Multithreading
مسؤول عنها OS	مسؤول عنها المبرمج.
التعامل معها أقل فاعلية لأن الانتقال من برنامج إلى برنامج آخر يحتاج الى الانتقال من فضاء ذاكرة إلى آخر	التعامل معها فعال أكثر (عمليات الانتقال أسهل)
كل برنامج يمثل process	ال thread هو جزء من ال process
ال OS (نظام التشغيل) يحجز ذاكرة مستقلة لكل process	يتم تشارك الذاكرة نفسها بين ال threads التي تنتمي لل process ذاتها
Heavyweight عملية التبديل تحتاج إلى وقت	Lightweight عملية التبديل سريعة

تعطل process لا يؤثر على بقية الـ process لأنها
مستقلة عن بعضها

إذا تعطل الـ thread قد يؤثر على بقية الـ thread لأنها
من الممكن أن يكون عملها مرتبط ببعضه حيث العديد
من الـ threads هي تتبع لـ process واحد (غير
مستقلة)

وكيف نطبق الـ thread برمجياً ؟

1. طريقة أولى : extends Thread class

2. طريقة ثانية : implements Runnable interface

ملاحظات :

(1) كل thread في الجافا هو object، فعند وجود عدة threads فكأنما لدي مجموعة من
الـ objects التي نقوم بـ switching بينها.

(2) عند وراثة الـ Thread class و هو موجود في java.lang فلا يحتاج إلى import
كل من الـ Thread class و الـ Runnable interface يحتوي على run Method وهي تعتبر العقل
المدير للـ thread ، وتكتب بالشكل: public void run() ، ونضع فيها الـ logic الذي نريده للـ thread ، وهي
تعتبر مثل الـ main في برنامج الجافا .

(3) Thread class يحتوي على العديد من الميثود ولكن Runnable interface لا يحتوي إلا على
الميثود run لذلك يدعى بـ functional interface

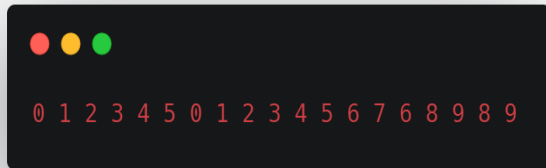
الطريقة الأولى عن طريق الـ thread class (a class extends Thread class):

```
1 public class Mythread extends Thread {
2     public void run() {
3         for (int i = 0; i < 10; i++)
4             System.out.print(i);
5     }
6 }
7
8 class Main {
9     public static void main(String[] args)
10    {
11        Mythread m = new Mythread();
12        Mythread m1 = new Mythread();
13        m.start();
14        m1.start();
15    }
```

الشرح:

هذا البرنامج يقوم بطباعة الاعداد من 0 ← 9
أولاً قمنا بإنشاء الكلاس Mythread حيث يرث من
الكلاس thread وتم استدعاؤه من دون import
لأنه يوجد تلقائياً في java.lang
تم خلق كائن من الـ Mythread class أي تم خلق
thread وحجز مكان في الذاكرة له -نخلقه كما نخلق
أي object من أي class- ولكنه لم يعمل بعد وعند
استدعاء start Method يصبح الـ Thread قابلاً
للتشغيل ولكن ليس بالضرورة أن يعمل مباشرة، قد
يأخذ بعض الوقت ليعمل وذلك حسب الأولوية
المعطاة له -سستحدث عن الأولويات بين الـ threads
لاحقاً- وتقوم الميثود start باستدعاء الميثود run
(ممنوع استدعاء الـ run مباشرة) وفي التابع run
تقوم بطباعة الأعداد

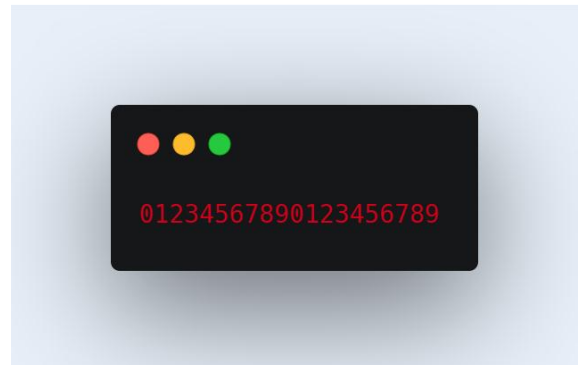
Output :



نلاحظ أنه تم طباعة الأعداد بشكل غير مرتب، أي تنفذ أحد الـ 2 threads لفترة ولكن لم ينتهي تنفيذه بعد حيث حدث switching بين الـ threads ، فبدأ الـ thread الآخر بالتنفيذ لفترة وهكذا حتى ينتهي تنفيذ كليهما، ولحل هذه المشكلة أي إذا كنا نريد أحد الـ threads أن تنفذ حتى تنتهي تماماً من التنفيذ دون أن يقطعها thread آخر نستدعي التابع join، ووظيفته إيقاف مجموعة الـ threads الباقية عن التنفيذ حتى ينتهي تنفيذ الـ thread الذي استدعاه تماماً، ومن ثم يبدأ الباقي بالتنفيذ حسب الدور.

من المثال السابق:

Output :



نلاحظ استدعاء كل من الـ thread m والـ m1 الـ thread لا join method بعد استدعاءهما لا start method والتغيير الذي حصل للخروج حيث تنفذ الـ thread في البداية وانتهى تنفيذه بالكامل ثم تنفذ الـ thread m1 وانتهى تنفيذه بالكامل أيضاً ، وأضافنا InterruptedException لأنه في حال تمت مقاطعة الشريد الذي ينتظر الشريد الآخر (الذي استدعى التابع sleep) سيحدث exception من النوع InterruptedException

```

1 import java.util.*;
2 class Main{
3     public static void main (String []
4 args)throws InterruptedException{
5         Mythread m=new Mythread();
6         Mythread m1=new Mythread();
7         m.start();
8         m.join();
9         m1.start();
10        m1.join();
11    }

```

ملاحظات:

- قمنا بوضع throws مع InterruptedException لنعلم بأن هذه الدالة من الممكن أن تثير استثناء من نوع InterruptedException ، وغالباً يتم رميه عندما يتم انقضاء وقت الانتظار بعملية معينة قبل انتهاء تنفيذها بسبب انقطاع الوصول إلى الصف الحالي (مثل عند استخدام تابع الـ join في جافا).
- من الممكن القيام بعملية الـ handling كالتالي:

```
public class Mythread extends Thread {  
    public void run() {  
        for (int i = 0; i < 10; i++)  
            System.out.print(i);  
    }  
}  
  
import java.util.*;  
class Main{  
    public static void main (String [] args){  
        Mythread m=new Mythread();  
        m.start();  
        try{  
            m.join();  
        }  
        catch (InterruptedException e){  
            System.out.println(e);  
        }  
        Mythread m1=new Mythread();  
        m1.start();  
    }  
}
```

تذكروا أن كل الأحلام تتحقق يا أصدقاء
وأن الأحلام التي لم تتحقق تنازل عنها أصحابها

من المثال السابق:

```

1 class Main{
2 public static void main (String [] args)throws
3 InterruptedException{
4 Mythread m=new Mythread();
5 Mythread m1=new Mythread();
6 m.start();
7 System.out.println("this is thread ");
8 System.out.println(m.isAlive());
9 }
10 }

```

output :

```

this is thread
true
0 1 2 3 4 5 6 7 8 9

```

عند إضافة تعليمة طباعة و استدعاء التابع isAlive و عدم استدعاء التابع join تمت طباعة الجملة و من ثم طباعة true و ثم طباعة القيم من 0 ← 9

وفي حال استدعاء التابع join بعد السطر الخامس مباشرةً سيتم طباعة :

0123456789

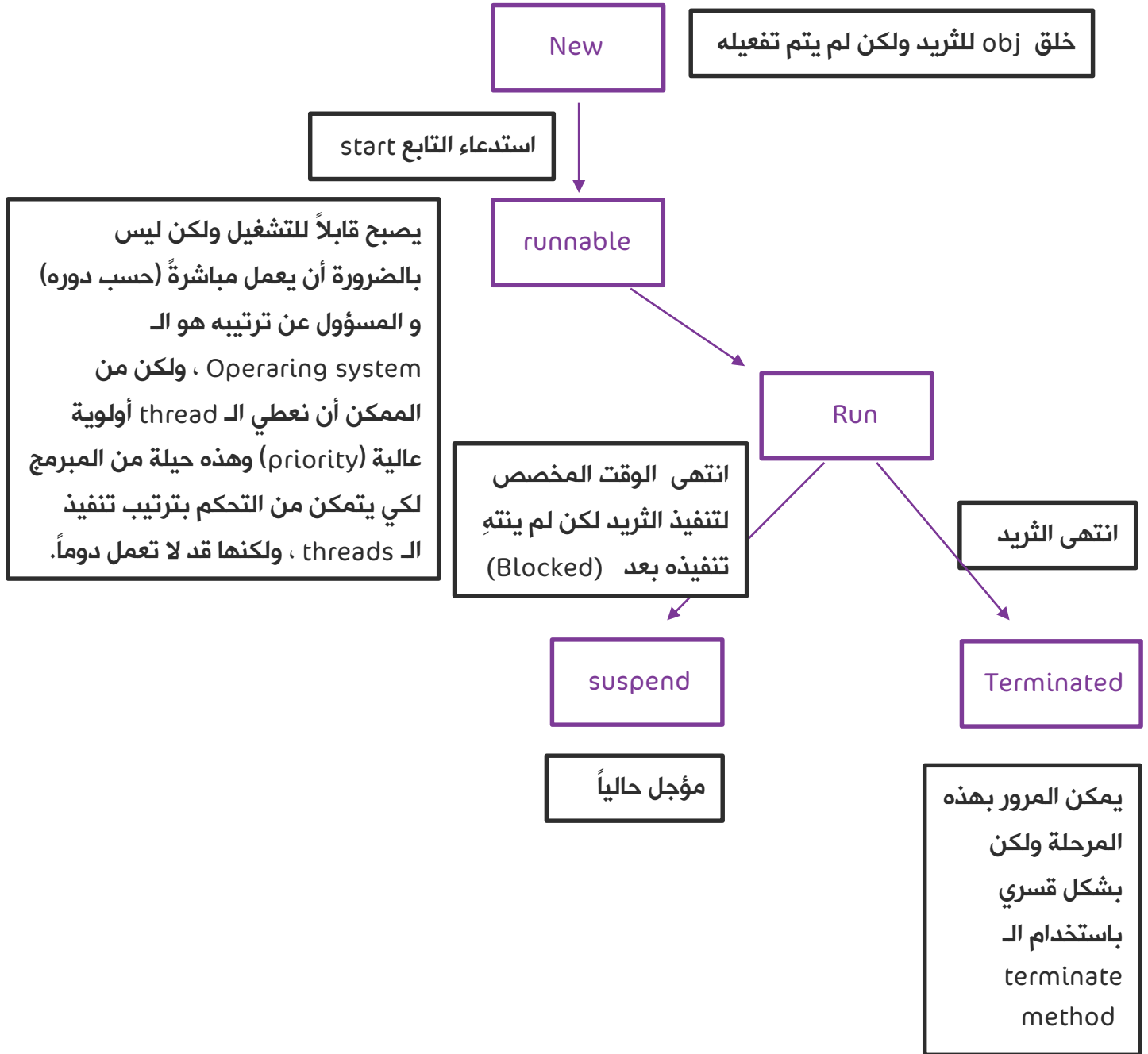
this is thread

false

ملاحظات:

- (1) القيام بـ override للتابع run() عند الوراثة من thread اختيارية ، حيث لن يتسبب عدم القيام بذلك بـ error ولكن لن نستفيد من الـ thread ؛ ففي هذه الحالة لن ينفذ الـ thread شيئاً لأن الـ run method الافتراضية الموجودة في الـ Thread class لا تقوم بتنفيذ أي شيء.
- (2) التابع isAlive هو تابع Boolean يعيد إما true أو false في حال كان الـ thread على قيد الحياة أم لا
- (3) التابع join يوقف الـ main عن التنفيذ حتى ينتهي تنفيذ الـ thread الذي استدعاه.
- (4) الدالة run() موجودة ضمن الـ Runnable ويمكن أن نقوم بـ override لها لنضع جميع الأوامر التي نريدها أن تنفذ عند تشغيل كائن الـ thread.
- (5) الميثود start() تقوم بتشغيل الـ run method وهي موجودة فقط داخل الكلاس thread ولا يجوز استدعاء الـ start() أكثر من مرة و إلا ستحصل على الـ illegalThreadStateException

Life cycle and states of thread in java



- ملاحظة أريسيزية: تم حذف الـ terminate method في الإصدارات الجديدة لجافا، فمن الممكن المرور بمرحلة الـ Terminated باستخدام الـ stop method.

إذن مراحل حياة ال thread بالترتيب كالتالي :

1. New state انشاء كائن لل thread ولم يتم تفعيله

2. Runnable state ندخل في هذه المرحلة عندما يتم استدعاء التابع start ، وهنا نميز حالتين :

Runnable جاهز للتشغيل في أي لحظة

Running قيد التشغيل بالفعل

3. waiting state : وتحدث عندما يستدعي ال thread ال wait method أو يستدعي thread آخر ال join method .

4. blocked state : دخل بمرحلة حظر (ال thread غير مؤهل للتنفيذ حالياً)

ملاحظة: في المرحلتين الثالثة والرابعة يكون ال thread غير نشط.

5. time waiting states : ال thread متوقف عن التنفيذ لمدة زمنية معينة، وتنتهي هذه المرحلة عندما:

3. 1) تنتهي مدة توقفه عن التنفيذ، مثل عند استدعاء ال: sleep method (long milliseconds) ، هذه ال method توقف ال thread عن التنفيذ لمدة معينة ندخل المدة ك parameter بال milliseconds

(2) تفعيله عن طريق تابع

6. terminated states : انتهاء تنفيذ الثريد بشكل سليم ، أي بهذه المرحلة تكون قد انتهت حياة الثريد

(أي إيقاف الكائن وحذفه من الذاكرة عن طريق ال interrupt method)

← توقفه بسبب خطأ ← غلط في التجزئة (قد يظهر عندما يحاول ال thread الوصول إلى منطقة في الذاكرة غير مسموح بالوصول إليها، مثل التعامل مع المؤشرات التي لم تتم تهيئتها أو تجاوز حدود مصفوفة ما)

← استثناء

← حالات أخرى (كالإنهاء الإجباري من المبرمج عن طريق method معينة)

ملاحظات :

■ نظام الجدولة (Scheduling System): النظام الذي يقرر أي خيط (thread) سيتم تشغيله في وقت معين ويعتمد على نظام التشغيل، وله نوعان:

1- جدولة زمنية: يتم تخصيص زمن صغير لكل ثريد، وبعد انتهاء الزمن ينتقل التحكم إلى ثريد آخر.

2- جدولة حسب الأولوية: يتم إعطاء الأولوية لل threads ذات الأولوية الأعلى (priority) ليتم تنبيهها.

■ هناك توابع تستخدم أيضاً (موجودة في ال Thread class) :

(1) isAlive : يرجع إما true أو false

(2) setName : يستطيع المبرمج تسمية ال thread كما يريد مثل thread4 ، threads

(3) **getName** : تقوم بإرجاع اسم ال thread مثل thread , thread
 (4) **getID** : تقوم بإرجاع الرقم الخاص بال thread
 (5) **currentThread** : تقوم بإرجاع ال object الخاص بال thread الذي يتم تنفيذه حالياً أي عند استدعاء هذه ال method بالشكل: **Thread.currentThread()**;
 (6) **sleep(long milliseconds)** : توقف تنفيذ أوامر كائن الخيط الذي قام باستدعائها لمدة محددة، وبعد انتهاء هذه المدة يعود للعمل من جديد.
 (causes the currently executing thread to sleep (temporarily cease execution) for the specified number of milliseconds.)

milliseconds هي المدة التي سيتوقف فيها كائن الخيط
 ترمي الاستثناء: (1) **IllegalThreadStateException** : في حال تم إعطاؤها قيمة أصغر من 0.
 (2) **InterruptedException** : في حال تم إيقاف كائن في فترة انتظاره.
 (7) **yield()** : للإشارة إلى أن ال thread الحالي مستعد للتخلي عن وحدة المعالجة المركزية مؤقتاً وإتاحة الفرصة لـ threads أخرى ذات أولوية مساوية أو أعلى للتنفيذ.
 (causes the currently executing thread object to temporarily pause and allow other threads to execute.)

(8) **notify()** : لتنبيه thread واحد ينتظر على نفس كائن المراقبة (monitor) باستخدام التابع **wait()**.
 (notifies a thread that is waiting for a condition that the condition has occurred. This is a method of the Object class and must be called from within a synchronized method or block.)

(9) **Wait()** : إذا كان هناك عدة threads تنتظر نفس الكائن باستخدام التابع **wait()** فسيتم اختيار thread واحد عشوائياً (أو وفقاً لنظام الجدولة - Scheduling System) ليتم تنبيهه، ال thread المختار ينتقل من حالة الانتظار (Waiting state) إلى حالة الجاهزية (Runnable state) لكنه لن يبدأ التنفيذ إلا عند حصوله على القفل الخاص بالكائن (synchronization) التي تعني أن خيط واحد فقط يمكنه العمل خلال وقت معين بشكل كامل، أي أن الخيط يحصل على قفل عليه بالنسبة لباقي ال threads).
 (causes current thread to wait for a condition to occur (another thread invokes the notify() method or the notifyAll() method for this object). This is a method of the Object class and must be called from within a synchronized method or block.)

(10) **notifyAll()** : لتنبيه جميع ال threads التي تنتظر على نفس كائن المراقبة (monitor - الكائن المرتبط دائماً بالكائن الذي تتم مزامنته باستخدام كلمة **synchronized**).

(11) **interrupt()** : ترسل إشارة إلى ال JVM لمقاطعة عمل كائن الخيط في حال كان يعمل، وفي حال كان كائن الخيط متوقفاً لمدة بسبب إحدى الدوال مثل **join()**, **wait()**, **Sleep()** فإنها لا تؤثر عليه أبداً.

■ عند إنشاء أول object من ال thread فإن الاسم الافتراضي له هو **thread0** والثاني **thread1** وهكذا، أما في حال تم إرسال اسم في ال constructor عند خلق ال thread فإنه يأخذ هذا الاسم وسيتم ترتيب الأرقام.

■ مفهوم ال priority :

Priorities are represented by a number between 1 and 10
كل thread له priority (أولوية) وهي تتراوح بين ال 0 و 10 ، وكل thread تلقائياً (by default) يأخذ قيمة ال priority متوسطة وهي (5) وفي حال تم استدعاء التابع getPriority لأي thread سيعطينا القيمة 5 ، ولكن سنميز قيمة ال priority:

MIN_PRIORITY 1 ←

NORM_PRIORITY 5 ←

MAX_PRIORITY 10 ←

حيث أن الكائن الذي يملك priority أعلى يتم تنفيذ أوامره في البداية قبل بقية ال threads ، ويمكن التلاعب بهذا الرقم فقط في حال كان لديك thread أهم من غيره في التنفيذ، وفي حال تم تعيين قيمة غير صالحة ل priority سيتم إظهار IllegalArgumentException

■ توابع تتعلق بال priority:

Public final int setPriority(int newPriority): تستخدم لتغيير قيمة ال Priority المعطاة لل thread
int final Public getPriority(): يعيد قيمة ال Priority لل thread الذي استدعاه

■ في ال OS هناك خوارزميتان لجدولة المعالجة، منها :

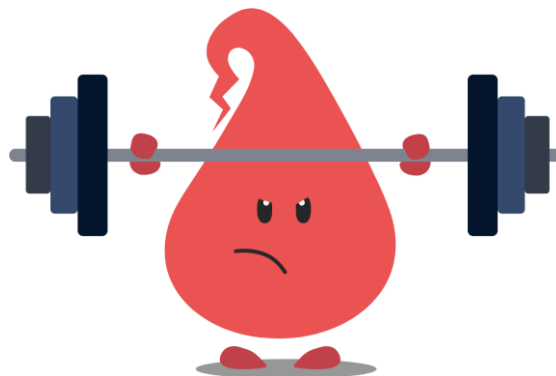
(1) First In First Out (FIFO) هذه الخوارزمية تعتمد على أن أول عنصر يدخل هو أول عنصر يخرج ، مثل ال Queue تأخذ بهذا المبدأ

(2) Last In First Out (LIFO) أي أن آخر عنصر يدخل هو أول عنصر يخرج
أمثلة : ال stack تأخذ بمبدأ LIFO

Round robin:

هو أحد أنماط جدولة المعالج في أنظمة التشغيل وفي جدولة Round Robin يتم تخصيص فترات زمنية معينة (time quantum) لكل عملية في النظام بشكل حلقي حيث يتم تنفيذ كل عملية للفترة المحددة ثم التبديل إلى العملية التالية وفي حال لم تنتهي العملية من التنفيذ خلال الفترة المحددة يتم وضعها في ال queue بالترتيب ليعاد تنفيذها بذات الطريقة (إعطاء فترة محدد للتنفيذ والانتقال للعملية التالية..).

مثل إعطاء كل thread 20 ثانية للتنفيذ وعندما تنتهي المدة ينتقل إلى ال thread الثاني وبالدور



Don't give up

```

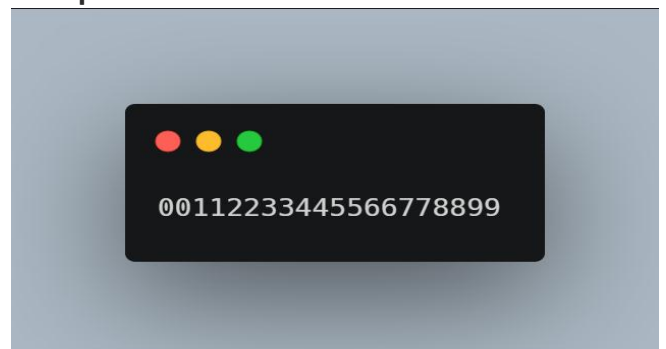
1 class MyThread implements Runnable {
2     public void run() {
3         for (int i = 0; i < 10; i++){
4             System.out.println(i);
5         }
6     }
7 }
8 Thread.sleep(1000000000);
9 }
10 catch(Exception e)
11 {
12     System.out.print(e);
13 }
14 }
15 }
16 public class Test {
17     public static void main(String[] args) {
18         Runnable r1 = new MyThread();
19         Runnable r2 = new MyThread();
20         Thread t1 = new Thread(r1);
21         Thread t2 = new Thread(r2);

```

تم إنشاء الـ class MyThread وهو يقوم بـ implements للـ Runnable وعندما أردنا استخدام الـ start قمنا بإنشاء obj من الكلاس Thread - حيث جعلنا Runnable reference يشير إلى object من الـ MyThread class (t1) - وتعرير الـ obj لـ constructor الـ Thread ومن ثم استدعاء التابع start باستخدام الـ obj الذي تم إنشاؤه من الـ Thread وسيقوم بتنفيذ الـ run الموجودة داخل الكلاس الذي ينفذ الانتريفييس مع تأمين 500 ميلي ثانية بين كل طباعة.

ملاحظة: عند استخدام التابع sleep يتوجب علينا استخدام try و catch

Output:



Synchronization in java:

- Multi threaded programs may often come to a situation where multiple Threads try to access the same resources

تستخدم للتأكد من أن thread واحد فقط قادر على الوصول إلى التابع أو block معينة وذلك من بداية عمل الـ thread إلى نهايته مما يؤدي إلى منع دخول بقية الـ Threads إلى التابع حتى ينتهي الـ Threads الحالي منه و لتحقيق ذلك نضع الـ Synchronization keyword قبل التابع أو البوك المراد استخدامه ، نقوم بكتابته

```
Synchronized(obj) { }
```

أي باختصار عندما تكون هناك threads تستخدم نفس الميثود (نفس الموارد) ستكون No safe method لذلك نتبع هذه الطريقة بجعلها safe .

```

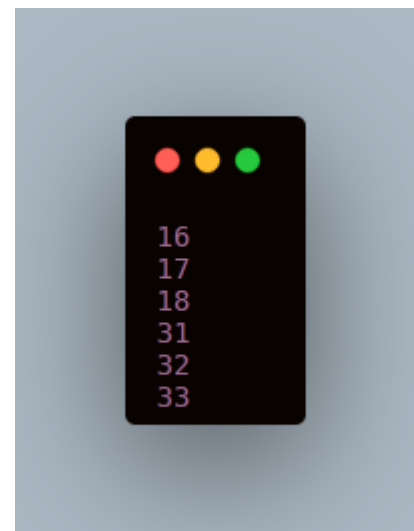
1 import java.io.*;
2 class Test {
3     synchronized void test_function(int n)
4     {
5         // synchronized method
6         for (int i = 1; i <= 3; i++) {
7             System.out.println(n + i);
8             try {
9                 Thread.sleep(500);
10            }
11            catch (Exception e) {
12                System.out.println(e);
13            }
14        }
15    }
16}
17 public class GFG {
18     // Main function
19     public static void main(String args[])
20     {
21         // only one object
22         final Test obj = new Test();
23
24         Thread a = new Thread() {
25             public void run() {
26                 obj.test_function(15);
27             }
28
29             Thread b = new Thread() {
30                 public void run() {
31                     obj.test_function(30);
32                 }
33             }
34 }

```

الشرح:

في هذا المثال استخدمنا طريقة Synchronized method لضمان أن thread واحد (خيطة) سيصل الى الـ method test_function ، وسيتم تنفيذه بالكامل بشكل آمن، وداخل هذه الميثود تم تعريف حلقة for مع تأمين 500 ميلي ثانية بين كل طباعة، وفي الـ main تم إنشاء كائنين a و b وعند استدعاء الـ start سيحاول كل من a و b استدعاء الميثود Test وفي نفس الوقت، ولأن هذه الميثود تم تعريفها Synchronized فسيقوم أحد الـ threads بالوصول إليها، ويتم حفظ الآخر حتى ينتهي الأول من تنفيذها لعدم حدوث تعارض أو مشاكل عند الطباعة

Output:



The End